

計算ゼミ 課題 3

石川曹太

2026 年 7 月 1 日

1 コード

Listing 1 メインプログラム

```
1 program ex03
2   use, intrinsic :: iso_fortran_env
3   use mod_linear_solver_lu
4
5   implicit none
6
7   real(real64), allocatable :: A(:, :), Ainv(:, :)
8
9   integer :: i
10
11  !使い分け
12  integer :: n = 3
13  !integer :: n = 2
14
15  allocate (A(n, n))
16  allocate (Ainv(n, n))
17
18  !使い分け
19  A = reshape([1.0d0, -2.0d0, 0.0d0, 1.0d0, 0.0d0, 2.0d0, -1.0d0, 1.0d0, 1.0
    d0], [n, n])
20  !A = reshape([0.0d0, 2.0d0, 1.0d0, 1.0d0, 3.0d0, 0.0d0, 0.0d0, 1.0d0, 2.0d0
    ], [n, n])
21  !A = reshape([0.0d0, 2.0d0, 1.0d0, 3.0d0], [n, n])
22
23  print *, "A ="
24  do i = 1, n
25      write(*, '(*(F25.15,1X))') A(i, :)
26  end do
27
28  call matrix_inverse(A, Ainv)
29
30
31  print *, "Ainv ="
```

```

32     do i = 1, n
33         write(*, '(*(F25.15,1X))') Ainv(i, :)
34     end do
35
36     deallocate (A, Ainv)
37
38
39 end program ex03

```

Listing 2 線形ソルバーモジュール

```

1 module mod_linear_solver_lu
2
3     use ,intrinsic :: iso_fortran_env
4     implicit none
5     private
6     public :: matrix_inverse
7
8 contains
9
10 !-----
11 ! 逆行列計算
12 !-----
13     subroutine matrix_inverse(A, Ainv)
14
15         real(real64), intent(in) :: A(:, :)
16         real(real64), intent(out) :: Ainv(:, :)
17
18         real(real64), allocatable :: P(:, :), L(:, :), U(:, :), Linv(:, :),
19             Uinv(:, :)
20         integer :: n, m
21
22         n = size(A, 1)
23         m = size(A, 2)
24
25         !それぞれの行列にサイズを決定
26         allocate(P(n, m), L(n, m), U(n, m), Linv(n, m), Uinv(n, m))
27
28         call LUdecomposition(P, A, L, U)
29         call Linverse(L, Linv)
30         call Uinverse(U, Uinv)
31
32         !Ainv = Uinv*Lin*P
33         Ainv = matmul(Uinv, matmul(Linv, P))
34
35         !数値計算ではallocateしたらdeallocateするのが習慣らしい
36         deallocate(P, L, U, Linv, Uinv)
37
38     end subroutine

```

```

38
39
40 ! -----
41 ! LU分解
42 ! -----
43
44     subroutine LUdecomposition(P, A, L, U)
45
46         real(real64), intent(in) :: A(:, :)
47         real(real64), intent(out) :: P(:, :), L(:, :), U(:, :)
48
49         ! nは行列のサイズ, i はPとLの初期値として1を導入する, kはピボット操作の回数
50         integer :: n, i, k
51
52         n = size(A, 1)
53
54         ! Aは固定し, Uを動かして最終的に  $PA = LU$  を目指す
55         U = A
56
57         ! PとLは最初に全成分を0にしてから i を用いて対角成分に1を挿入
58         P = 0.0d0
59         L = 0.0d0
60         do i = 1, n
61             P(i, i) = 1.0d0
62             L(i, i) = 1.0d0
63         end do
64
65         do k = 1, n-1
66
67             call Pivot(P, L, U, k)
68             call GaussElimination(L, U, k)
69
70         end do
71
72     end subroutine LUdecomposition
73
74
75 ! -----
76 ! ピボット選択
77 ! -----
78
79     subroutine Pivot(P, L, U, k)
80
81         real(real64), intent(inout) :: P(:, :), L(:, :), U(:, :)
82         integer, intent(in) :: k
83
84         ! nは行列Uのサイズ, i はピボットとなる最大絶対値を探索するループで使う, pivot_rowは最終的にk行と入れ替える行
85         integer :: n, i, pivot_row

```

```

86
87      !maxは探索する中での最大値, Ozは入れ替えを行う際基準となるk行を1行分丸ごと保存
88      real(real64) :: max, Pz(size(P, 2)), Lz(size(L, 2)), Uz(size(U, 2))
89
90      n = size(U, 1)
91      pivot_row = k
92      max = abs(U(k, k))
93
94      do i = k+1, n
95
96          if (abs(U(i, k)) > max) then
97              max = abs(U(i, k))
98              pivot_row = i
99          end if
100
101      end do
102
103      !Uのピボットを見つけたら合わせてP, L, Uの成分を入れ替える
104      !ただし, Lに関しては確定した対角成分以外を入れ替えるため発動する条件を付与する
105
106      if (pivot_row /= k) then
107
108          Pz = P(pivot_row, :)
109          P(pivot_row, :) = P(k, :)
110          P(k, :) = Pz
111
112          Uz = U(pivot_row, :)
113          U(pivot_row, :) = U(k, :)
114          U(k, :) = Uz
115
116      end if
117
118      if (k > 1) then
119
120          Lz(1:k-1) = L(pivot_row, 1:k-1)
121          L(pivot_row, 1:k-1) = L(k, 1:k-1)
122          L(k, 1:k-1) = Lz(1:k-1)
123
124      end if
125
126      end subroutine Pivot
127
128
129      !-----
130      ! ガウス消去
131      !-----
132      subroutine GaussElimination(L, U, k)
133
134      real(real64), intent(inout) :: L(:, :), U(:, :)

```

```

135     integer, intent(in) :: k
136
137     ! nは行列のサイズ, 行列Uの成分をU ( i , j )とし, それぞれの成分で計算を行う
138     integer :: n, m, i, j
139     ! factorはピボット選択後ガウス消去を行うために行以外の行に掛ける係数k
140     real(real64) :: factor
141
142     n = size(U, 1)
143     m = size(U, 2)
144
145     do i = k+1, n
146
147         factor = U(i, k) / U(k, k)
148         ! factorが決定すると行列Lの下三角の成分が決定する
149         L(i, k) = factor
150
151         do j = 1, m
152
153             U(i, j) = U(i, j) - U(k, j) * factor
154
155         end do
156
157     end do
158
159     end subroutine GaussElimination
160
161
162     ! -----
163     ! Lの逆行列計算 (下三角) 前進代入
164     ! -----
165     subroutine Linverse(L, Linv)
166
167         real(real64), intent(in) :: L(:, :)
168         real(real64), intent(out) :: Linv(:, :)
169
170         ! nは行列Lのサイズ, Lの成分をL ( i , j )とする
171         integer :: n, m, i, j, k
172         ! s (sum) は一時的に和を保存して, 最終的に-L ( j , j )で割る 数値計算でよく用いる手法
173         real(real64) :: s
174
175         n = size(L, 1)
176         m = size(L, 2)
177
178         ! Linvの中身は後から挿入するとして, 一旦全成分を0で統一
179         Linv = 0.0d0
180
181         ! 1列目から考える
182         do j = 1, m
183

```

```

184         Linv(j, j) = 1.0d0 / L(j, j)
185
186         !上では対角成分を確定させたから、対角以下の成分を考える
187         do i = j+1, n
188
189             s = 0.0d0
190
191             do k = 1, i-1
192
193                 s = s + L(i, k) * Linv(k, j)
194
195             end do
196
197             Linv(i, j) = -s / L(i, i)
198
199         end do
200
201     end do
202
203     end subroutine Linverse
204
205
206     !-----
207     ! Uの逆行列計算（上三角）後退代入
208     !-----
209     subroutine Uinverse(U, Uinv)
210
211         real(real64), intent(in) :: U(:, :)
212         real(real64), intent(out) :: Uinv(:, :)
213
214         integer :: n, m, i, j, k
215         real(real64) :: s
216
217         n = size(U, 1)
218         m = size(U, 2)
219
220         Uinv = 0.0d0
221
222         !後退代入なので i, j は後ろから考える
223
224         do j = m, 1, -1
225
226             Uinv(j, j) = 1.0d0 / U(j, j)
227
228             do i = j-1, 1, -1
229
230                 s = 0.0d0
231
232                 do k = i+1, j

```

```

233
234             s = s + U(i, k) * Uinv(k, j)
235
236         end do
237
238         Uinv(i, j) = -s / U(i, i)
239
240     end do
241
242 end do
243
244 end subroutine Uinverse
245
246
247 end module mod_linear_solver_lu

```

2 LU 分解の説明